

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

ОТЧЕТ
О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №3
ПО ДИСЦИПЛИНЕ
«ОСНОВЫ КОМПЬЮТЕРНОГО МОДЕЛИРОВАНИЯ
ДИНАМИЧЕСКИХ СИСТЕМ»
ВАРИАНТ ЗАДАНИЯ №25

Выполнил студент группы М8О-206Б-23
Сергеев В. С. _____

подпись, дата

Проверил и принял

Ст. преп. каф. 802 Волков Е.В. _____

подпись, дата

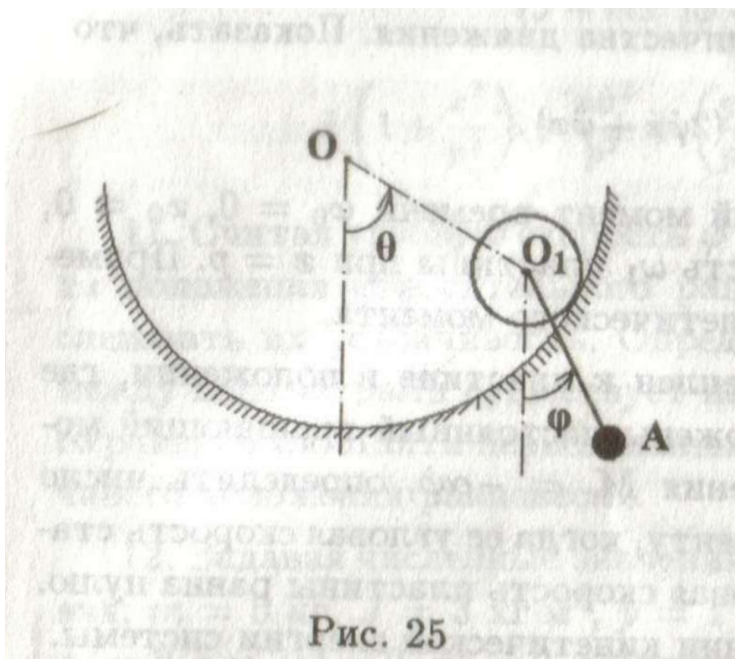
с оценкой _____

Вариант №25

Задание:

Проинтегрировать систему дифференциальных уравнений движения системы с двумя степенями свободы с помощью средств Python. Построить анимацию движения системы, а также графики законов движения системы и указанных в задании реакций для разных случаев системы.

Механическая система:



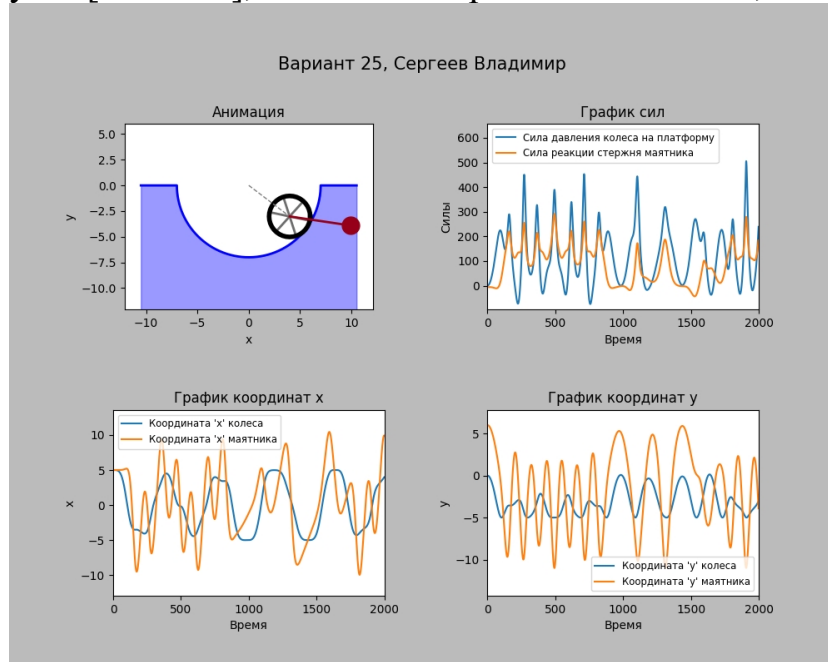
Текст программы

Текст программы в приложении №1.

Результат работы программы:

1. $m_1 = 10$; $m_2 = 2$; $r = 2$; $R = 7$; $l = 6$; $g = 9.80665$;

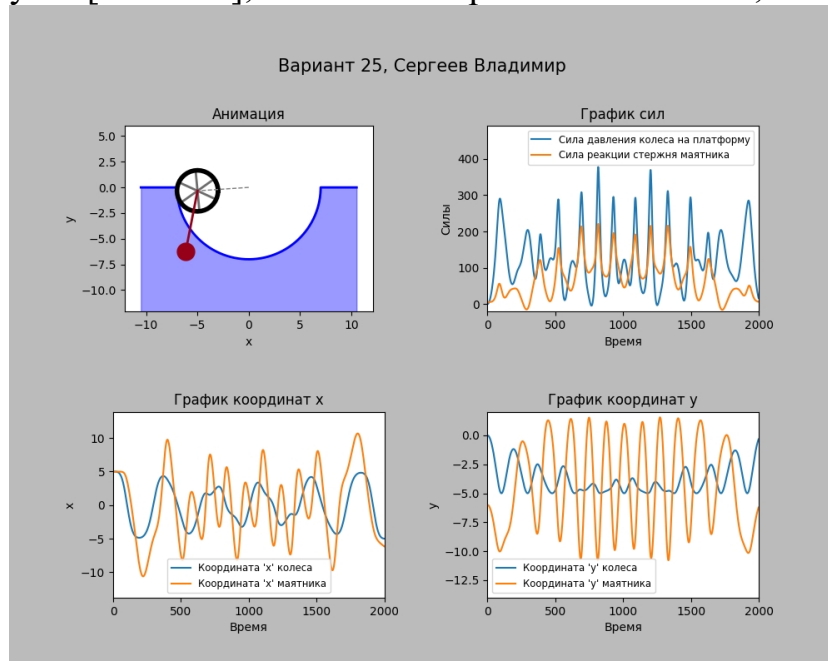
$y_0 = [\pi/2 \ 0 \ \pi \ 0]$; – колесо в верхнем положении, маятник в верхнем положении:



Результат: сначала колесо катается с небольшой амплитудой, а маятник вращается примерно в 2 раза быстрее. Через два полных колебания амплитуда колеса резко увеличивается, а частота вращения маятника начинает почти равняться частоте колеса. Через еще одно полное колебание амплитуда колеса снова уменьшается, а частота маятника увеличивается вдвое.

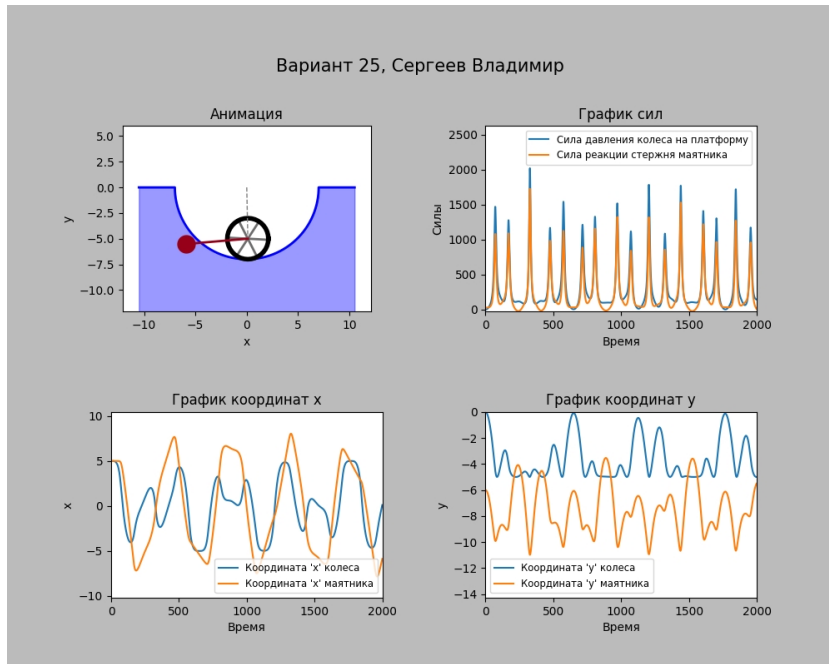
2. $m_1 = 10$; $m_2 = 2$; $r = 2$; $R = 7$; $l = 6$; $g = 9.80665$;

$y_0 = [\pi/2 \ 0 \ 0 \ 0]$; – колесо в верхнем положении, маятник в нижнем положении:



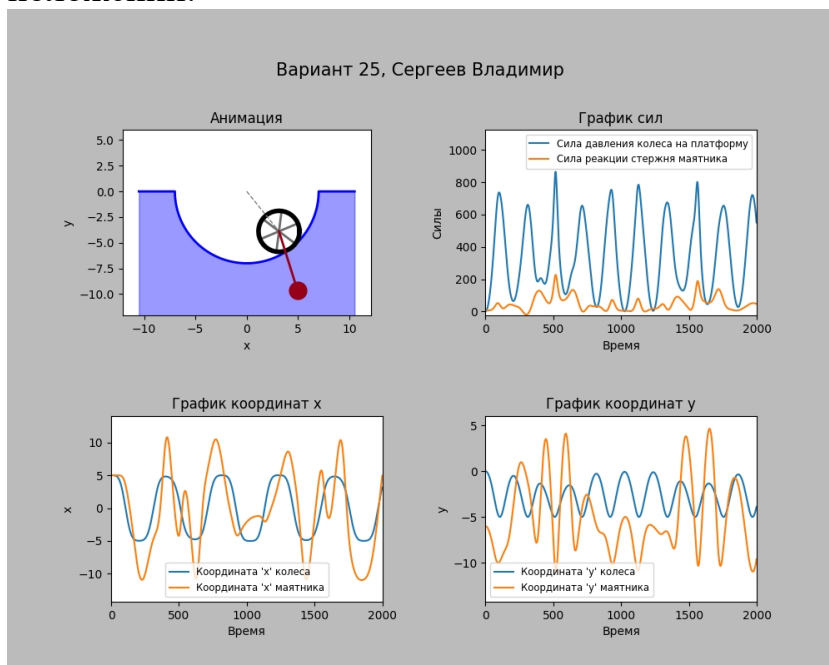
Результат: амплитуда колебаний колеса при неизменной частоте качения сначала плавно уменьшается, а затем снова увеличивается. Амплитуда и частота колебаний маятника сначала увеличивается, а затем снижается.

3. $m_1 = 10; m_2 = 20; r = 2; R = 7; l = 6; g = 9.80665;$
 $y_0 = [\pi/2 \ 0 \ 0 \ 0];$ – тяжелый маятник, колесо в верхнем положении, маятник в нижнем положении:



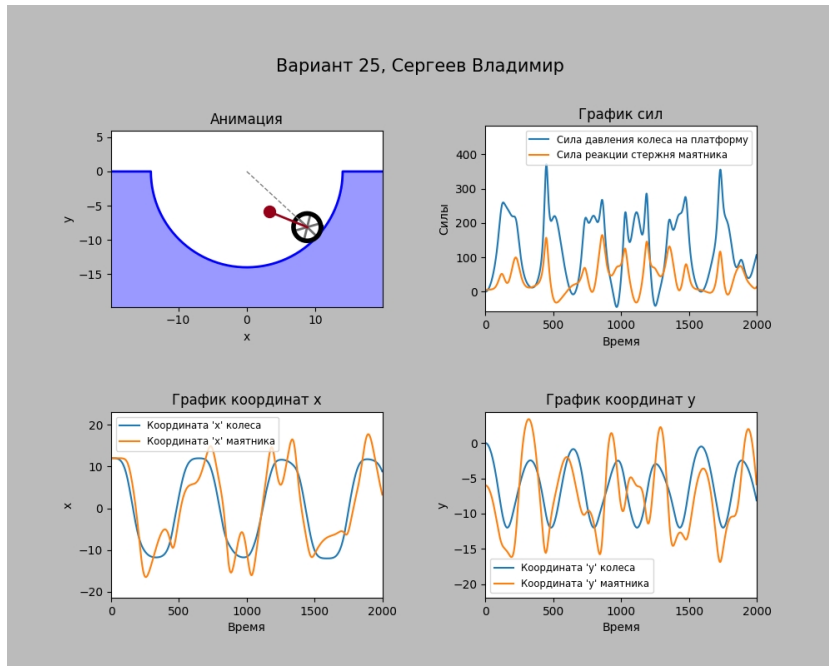
Результат: колесо медленно неравномерно катается, маятник качается с постоянной частотой.

4. $m_1 = 30; m_2 = 2; r = 2; R = 7; l = 6; g = 9.80665;$
 $y_0 = [\pi/2 \ 0 \ 0 \ 0];$ – тяжелое колесо в верхнем положении, маятник в нижнем положении:



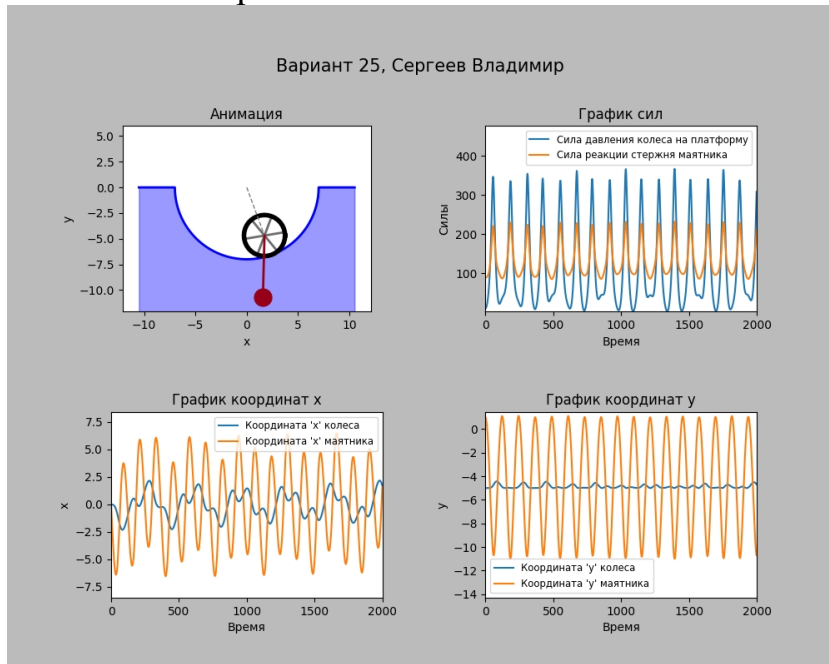
Результат: Колесо катается с постоянной амплитудой и частотой. Маятник сначала качается с небольшой амплитудой, затем амплитуда уменьшается, и снова увеличивается.

5. $m_1 = 10; m_2 = 2; r = 2; R = 14; l = 6; g = 9.80665;$
 $y_0 = [\pi/2 \ 0 \ 0 \ 0];$ – полость большого радиуса, колесо в верхнем положении, маятник в нижнем положении:



Результат: Колесо катается с постоянной частотой и амплитудой. Сначала маятник качается с небольшой амплитудой, амплитуда увеличивается, затем уменьшается, и снова увеличивается.

6. $m_1 = 10; m_2 = 2; r = 2; R = 7; l = 6; g = 9.80665;$
 $y_0 = [0 \ 0 \ \pi \ 3];$ – колесо в нижнем положении, маятник в верхнем положении с начальной скоростью:



Результат: колесо неравномерно катается с небольшой постоянной амплитудой и постоянной частотой, маятник вращается с постоянной частотой.

Вывод:

В ходе выполнения лабораторной работы была написана программа на языке Python, для решения заданных дифференциальных уравнений, описывающих движение системы тел и силы реакций, а также для симуляции системы и построения анимации движения системы и графиков физических величин.

Приложение №1

```
import numpy as np
from warnings import warn
from scipy.integrate import odeint
from matplotlib import use
from matplotlib.pyplot import figure, show
from matplotlib.gridspec import GridSpec
from matplotlib.animation import FuncAnimation
```

class Wheel:

"""Class for displaying a wheel."""

```
def __init__(self, radius, position, phase: any = 0, spokes_count=2, precision=360,
             tire_color='black', spokes_color='gray', tire_width=5, spokes_width=2):

    self.spokes_count = spokes_count
    self.radius = radius

    self.spoke_functions = (
        (lambda x, phase, number:
         np.array([
             x + self.radius * np.cos(phase + np.pi / self.spokes_count * number),
             x - self.radius * np.cos(phase + np.pi / self.spokes_count * number)
         ])),
        (lambda y, phase, number:
         np.array([
             y + self.radius * np.sin(phase + np.pi / self.spokes_count * number),
             y - self.radius * np.sin(phase + np.pi / self.spokes_count * number)
         ]))
    )

    self.spokes = []
    for i in range(self.spokes_count):
        self.spokes.append(animation_region.plot(
            self.spoke_functions[0](position[0], phase, i),
            self.spoke_functions[1](position[1], phase, i),
            color=spokes_color, lw=spokes_width
        )[0])

    angles = np.linspace(0, 2 * np.pi, precision + 1)
    self.tire_values = self.radius * np.cos(angles), self.radius * np.sin(angles)

    self.tire, = animation_region.plot(self.tire_values[0] + position[0], self.tire_values[1] + position[1],
                                       color=tire_color, lw=tire_width)

    def update(self, position, phase):
        for i in range(self.spokes_count):
            self.spokes[i].set_data(
                self.spoke_functions[0](position[0], phase, i),
                self.spoke_functions[1](position[1], phase, i)
            )

        self.tire.set_data(self.tire_values[0] + position[0], self.tire_values[1] + position[1])

    def return_plot(self):
        return *self.spokes, self.tire,
```

class Pendulum:

"""Class for displaying a pendulum."""

```
def __init__(self, suspension_position, weight_position,
             weight_radius=20, color='#960018', line_width=2):
    self.weight, = animation_region.plot(
        np.array([weight_position[0]]),
        np.array([weight_position[1]]),
        color=color, lw=line_width, marker='o', markersize=weight_radius
    )
    self.rod, = animation_region.plot(
        np.array([suspension_position[0], weight_position[0]]),
        np.array([suspension_position[1], weight_position[1]]),
        color=color, lw=line_width
    )
```

```

def update(self, weight_position, suspension_position):
    self.weight.set_data(
        np.array([weight_position[0]]),
        np.array([weight_position[1]])
    )
    self.rod.set_data(
        np.array([suspension_position[0], weight_position[0]]),
        np.array([suspension_position[1], weight_position[1]])
    )

def return_plot(self):
    return self.weight, self.rod,

class Cord:
    """Class for displaying a cord."""

    def __init__(self, start, end, color='gray', line_style='--', line_width=1):
        self.plot, = animation_region.plot(
            np.array([start[0], end[0]]),
            np.array([start[1], end[1]]),
            color=color, linestyle=line_style, lw=line_width
        )

    def update(self, start, end):
        self.plot.set_data([start[0], end[0]], [start[1], end[1]])

    def return_plot(self):
        return self.plot,

def make_concave_platform(radius, position, precision=180, color='blue'):
    """Function for displaying a concave platform."""

    angles = np.linspace(-np.pi, 0, precision)
    plot_values = (
        np.concatenate((
            [-1.5 * radius, -radius], radius * np.cos(angles), [radius, 1.5 * radius]
        )) + position[0],
        np.concatenate((
            [0, 0], radius * np.sin(angles), [0, 0]
        )) + position[1]
    )

    animation_region.plot(
        plot_values[0], plot_values[1],
        color=color, lw=2
    )

    animation_region.fill_between(
        plot_values[0],
        plot_values[1],
        np.full(precision + 4, -2 * radius),
        alpha=.4, color=color
    )

if __name__ == '__main__':
    # --- Simulation settings ---

    G = 9.80665 # Gravitational acceleration

    PLATFORM_POSITION = (0, 0) # Position of concave platform
    PLATFORM_RADIUS = 7 # Radius of concave platform

    WHEEL_RADIUS = 2 # Radius of wheel
    WHEEL_MASS = 10 # Mass of wheel

    PENDULUM_LENGTH = 6 # Length of pendulum rod
    PENDULUM_MASS = 2 # Mass of pendulum rod
    PENDULUM_RADIUS = 15 # Radius of pendulum weight, doesn't affect calculations.

    # Initial values of state variables
    INITIAL_CONDITIONS = [

        # Angle of wheel deviation
        np.pi * 4 / 13,

        # Angular velocity of wheel deviation

```

```

0,

# Angle of pendulum deviation
-np.pi * 2 / 3,

# Angular velocity of pendulum deviation
0
]

MAX_TIME = 60 # Maximum simulation time
STEPS: int = 2000 # Count of time points
VISUALIZATION_RATIO: int = 1 # Ratio of number of time points to total number of frames
FPS = 60 # Frames per second

# --- Calculations ---

if not isinstance(VISUALIZATION_RATIO, int):
    raise ValueError(
        f"Invalid type of VISUALIZATION_RATIO. The type must be an int, but type is {type(VISUALIZATION_RATIO)}."
    )
elif VISUALIZATION_RATIO <= 0 or VISUALIZATION_RATIO > STEPS:
    raise ValueError(
        f"Invalid value of VISUALIZATION_RATIO. The value must be between 1 and {STEPS}."
        f"But value is {VISUALIZATION_RATIO}."
    )

if not isinstance(STEPS, int):
    raise ValueError(
        f"Invalid type of STEPS. The type must be an int, but type is {type(STEPS)}."
    )

# Calculations for wheel are performed in such a way that wheel is on virtual rod.
WHEEL_VIRTUAL_ROD_LENGTH = PLATFORM_RADIUS - WHEEL_RADIUS

time_points = np.linspace(0, MAX_TIME, STEPS)

def wheel_and_pendulum(y, _):
    if y[0] < -np.pi / 2 or y[0] > np.pi / 2:
        warn(f"Non-physical state detected. Wheel angle must be between -pi/2 and pi/2 (wheel angle is {y[0]})")

    cos_of_difference = np.cos(y[2] - y[0])
    pendulum_torque_component = PENDULUM_MASS * PENDULUM_LENGTH
    matrix_a = np.array([
        [
            (3 * WHEEL_MASS / 2 + PENDULUM_MASS) * WHEEL_VIRTUAL_ROD_LENGTH,
            cos_of_difference * pendulum_torque_component
        ],
        [
            cos_of_difference * WHEEL_VIRTUAL_ROD_LENGTH,
            PENDULUM_LENGTH
        ]
    ])
    sin_of_difference = np.sin(y[2] - y[0])
    matrix_b = np.array([
        sin_of_difference * y[3] ** 2 * pendulum_torque_component
        - np.sin(y[0]) * G * (WHEEL_MASS + PENDULUM_MASS),
        -(sin_of_difference * y[1] ** 2 * WHEEL_VIRTUAL_ROD_LENGTH
        + np.sin(y[2]) * G)
    ])
    accelerations = np.linalg.solve(matrix_a, matrix_b)
    return y[1], accelerations[0], y[3], accelerations[1]

solution = odeint(func=wheel_and_pendulum, y0=INITIAL_CONDITIONS, t=time_points)

# Angle of wheel deviation at each time point
wheel_angles = np.array(solution[:, 0])

# Angular velocity of wheel at each time point
wheel_angular_velocities = np.array(solution[:, 1])

# Angular acceleration of wheel at each time point
wheel_angular_accelerations = np.gradient(wheel_angular_velocities, time_points)

# Angle of pendulum deviation at each time point
pendulum_angles = np.array(solution[:, 2])

# Angular velocity of pendulum at each time point

```

```

pendulum_angular_velocities = np.array(solution[:, 3])

# Angular acceleration of pendulum at each time point
pendulum_angular_accelerations = np.gradient(pendulum_angular_velocities, time_points)

shifted_wheel_angles = wheel_angles - np.pi / 2

# Wheel location at each time point
wheel_positions = (
    PLATFORM_POSITION[0] + WHEEL_VIRTUAL_ROD_LENGTH * np.cos(shifted_wheel_angles),
    PLATFORM_POSITION[1] + WHEEL_VIRTUAL_ROD_LENGTH * np.sin(shifted_wheel_angles)
)

GEAR_RATIO = -(WHEEL_VIRTUAL_ROD_LENGTH / WHEEL_RADIUS)

# Phase of rotation of the wheel at each time point
wheel_phases = wheel_angles * GEAR_RATIO

shifting_pendulum_angles = pendulum_angles - np.pi / 2

# Pendulum location at each time point
pendulum_positions = (
    wheel_positions[0] + PENDULUM_LENGTH * np.cos(shifting_pendulum_angles),
    wheel_positions[1] + PENDULUM_LENGTH * np.sin(shifting_pendulum_angles)
)

cos_of_difference = np.cos(pendulum_angles - wheel_angles)
sin_of_difference = np.sin(pendulum_angles - wheel_angles)

platform_reaction_force = (
    (WHEEL_MASS + PENDULUM_MASS)
    * (np.cos(wheel_angles) * G + WHEEL_VIRTUAL_ROD_LENGTH * wheel_angular_velocities ** 2)
    + PENDULUM_MASS * PENDULUM_LENGTH
    *
    (
        sin_of_difference * pendulum_angular_accelerations
        + cos_of_difference * pendulum_angular_velocities ** 2
    )
)

rod_reaction_force = (
    PENDULUM_MASS *
    (
        np.cos(pendulum_angles) * G
        +
        (
            cos_of_difference * wheel_angular_velocities ** 2
            - sin_of_difference * wheel_angular_accelerations
        )
        * WHEEL_VIRTUAL_ROD_LENGTH
        + pendulum_angular_velocities ** 2 * PENDULUM_LENGTH
    )
)

# --- Visualization ---

use('QtAgg')

window = figure(facecolor='#bbbbbb', figsize=(10, 8))
grid_specification = GridSpec(4, 3, height_ratios=[1, 10, 3, 10], width_ratios=[10, 1, 10])

title_region = window.add_subplot(grid_specification[0, 1])
title_region.axis('off')
title_region.text(0, 2, "Вариант 25, Сергеев Владимир",
    horizontalalignment='center', verticalalignment='center', fontsize=15)

def calculate_limits(**kwargs):
    space_ratio = 0.3
    max_value = np.max(np.array([*map(np.max, kwargs.values())]))
    min_value = np.min(np.array([*map(np.min, kwargs.values())]))

    if max_value > 0:
        upper_limiter = (1 + space_ratio) * max_value
    else:
        upper_limiter = (1 - space_ratio) * max_value

    if min_value > 0:
        lower_limiter = (1 - space_ratio) * min_value

```

```

else:
    lower_limiter = (1 + space_ratio) * min_value

return lower_limiter, upper_limiter

```

```

# Animation
animation_region = window.add_subplot(grid_specification[1, 0])
animation_region.set_title("Анимация")
animation_region.set_xlabel("x")
animation_region.set_ylabel("y")
animation_region_limit = 1.1 * (WHEEL_VIRTUAL_ROD_LENGTH + PENDULUM_LENGTH)
animation_region.set_xlim(-animation_region_limit, animation_region_limit)
animation_region.set_ylim(-animation_region_limit, PENDULUM_LENGTH)
# We will set the value to "image" in order to comply with the established limits and avoid distortions.
animation_region.axis('image')

```

```

# Forces
forces_region = window.add_subplot(grid_specification[1, 2])
forces_region.set_title("График сил")
forces_region.set_xlabel("Время")
forces_region.set_ylabel("Силы")
forces_region.set_xlim(0, STEPS)
forces_region.set_ylim(calculate_limits(platform=platform_reaction_force, rod=rod_reaction_force))

```

```

# X-coordinates
x_region = window.add_subplot(grid_specification[3, 0])
x_region.set_title("График координат x")
x_region.set_xlabel("Время")
x_region.set_ylabel("x")
x_region.set_xlim(0, STEPS)
x_region.set_ylim(calculate_limits(wheel=wheel_positions[0], pendulum=pendulum_positions[0]))

```

```

# Y-coordinates
y_region = window.add_subplot(grid_specification[3, 2])
y_region.set_title("График координат y")
y_region.set_xlabel("Время")
y_region.set_ylabel("y")
y_region.set_xlim(0, STEPS)
y_region.set_ylim(calculate_limits(wheel=wheel_positions[1], pendulum=pendulum_positions[1]))

```

```

# Animation
make_concave_platform(PLATFORM_RADIUS, PLATFORM_POSITION, precision=180)
wheel = Wheel(WHEEL_RADIUS, (wheel_positions[0][0], wheel_positions[1][0]), wheel_phases[0],
              spokes_count=3, precision=36, spokes_color='#6e6e6e', tire_width=4)
# Cord from platform position to wheel position
cord = Cord(PLATFORM_POSITION, (wheel_positions[0][0], wheel_positions[1][0]))
pendulum = Pendulum(
    (wheel_positions[0][0], wheel_positions[1][0]),
    (pendulum_positions[0][0], pendulum_positions[1][0]),
    weight_radius=PENDULUM_RADIUS
)

```

```

# Forces
platform_reaction_force_plot, = forces_region.plot([0], [0])
rod_reaction_force_plot, = forces_region.plot([0], [0])
forces_region.legend(
    [platform_reaction_force_plot, rod_reaction_force_plot],
    ["Сила давления колеса на платформу", "Сила реакции стержня маятника"],
    fontsize='small'
)

```

```

# X-coordinates
wheel_x_plot, = x_region.plot([0], [0])
pendulum_x_plot, = x_region.plot([0], [0])
x_region.legend(
    [wheel_x_plot, pendulum_x_plot],
    ["Координата 'x' колеса", "Координата 'x' маятника"],
    fontsize='small'
)

```

```

# Y-coordinates
wheel_y_plot, = y_region.plot([0], [0])
pendulum_y_plot, = y_region.plot([0], [0])
y_region.legend(
    [wheel_y_plot, pendulum_y_plot],
    ["Координата 'y' колеса", "Координата 'y' маятника"],
    fontsize='small'
)

```

```

steps_numbers_list = np.array(range(STEPS))

def animate(i):
    i *= VISUALIZATION_RATIO

    # Animation
    wheel.update(
        (wheel_positions[0][i], wheel_positions[1][i]),
        wheel_phases[i]
    )
    cord.update(
        PLATFORM_POSITION,
        (wheel_positions[0][i], wheel_positions[1][i])
    )
    pendulum.update(
        (pendulum_positions[0][i], pendulum_positions[1][i]),
        (wheel_positions[0][i], wheel_positions[1][i])
    )

    # Forces
    platform_reaction_force_plot.set_data(steps_numbers_list[i + 1], platform_reaction_force[i + 1])
    rod_reaction_force_plot.set_data(steps_numbers_list[i + 1], rod_reaction_force[i + 1])

    # X-coordinates
    wheel_x_plot.set_data(steps_numbers_list[i + 1], wheel_positions[0][i + 1])
    pendulum_x_plot.set_data(steps_numbers_list[i + 1], pendulum_positions[0][i + 1])

    # Y-coordinates
    wheel_y_plot.set_data(steps_numbers_list[i + 1], wheel_positions[1][i + 1])
    pendulum_y_plot.set_data(steps_numbers_list[i + 1], pendulum_positions[1][i + 1])

    return (*wheel.return_plot(), *cord.return_plot(), *pendulum.return_plot(),
            wheel_x_plot, pendulum_x_plot, wheel_y_plot, pendulum_y_plot, platform_reaction_force_plot,
            rod_reaction_force_plot)

ani = FuncAnimation(fig=window, func=animate, frames=STEPS // VISUALIZATION_RATIO, interval=round(1000 / FPS),
                    repeat=False, blit=True)
show()
print("Симуляция успешно завершена.")

```